ELECTRONIC ASSIGNMENT COVERSHEET

Student Number:	35517678
Surname:	New
Given name:	Wei Nern
Email:	35517678@student.murdoch.edu.au

Unit Code:	ICT283
Unit name:	Data Structures and Abstraction
Enrolment mode:	Internal
Date:	27/7/2025
Assignment number:	2
Assignment name:	
Tutor:	Leng Kang

Student's Declaration:

- Except where indicated, the work I am submitting in this assignment is my own work and has not been submitted for assessment in another unit.
- This submission complies with Murdoch University's academic integrity commitments. I am aware that information about plagiarism and associated penalties can be found at http://our.murdoch.edu.au/Educational-technologies/Academic-integrity/. If I have any doubts or queries about this, I am further aware that I can contact my Unit Coordinator prior to submitting the assignment.
- I acknowledge that the assessor of this assignment may, for the purpose of assessing this assignment:
  - reproduce this assignment and provide a copy to another academic staff member; and/or
  - submit a copy of this assignment to a plagiarism-checking service. This web-based service may retain a copy of this work for the sole purpose of subsequent plagiarism checking, but has a legal agreement with the University that it will not share or reproduce it in any form.
- I have retained a copy of this assignment.
- I will retain a copy of the notification of receipt of this assignment. If you have not received a receipt within three days, please check with your Unit Coordinator.

I am aware that I am making this declaration by submitting this document electronically and by using my Murdoch ID and password it is deemed equivalent to executing this declaration with my written signature.
Optional Comments to Tutor: *e.g. If this is a group assignment, list group members here*

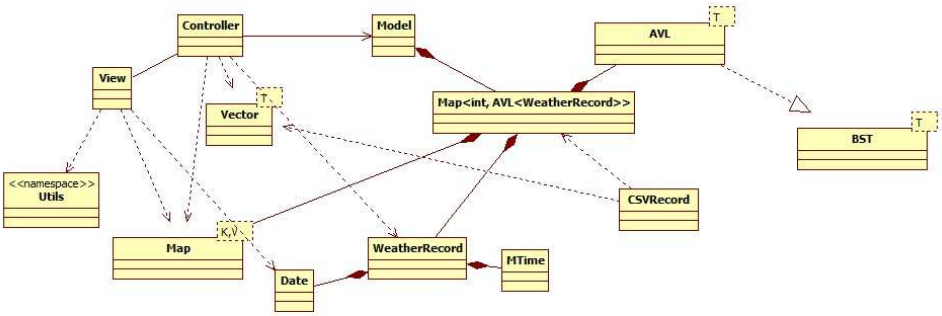
*If you can, please insert this completed form into the body of **each** assignment you submit. Follow the instructions in the Unit Information and Learning Guide about how to submit your file(s) and how to name them, so the Unit Coordinator knows whose work it is.*

Start your assignment on the next page.

Contents

UML Diagram

Below is a high-level diagram for assignment 2. The structure is modified to include CSVRecord, Map, AVL, and BST. Additionally, a namespace called ‘Utils’ is created.

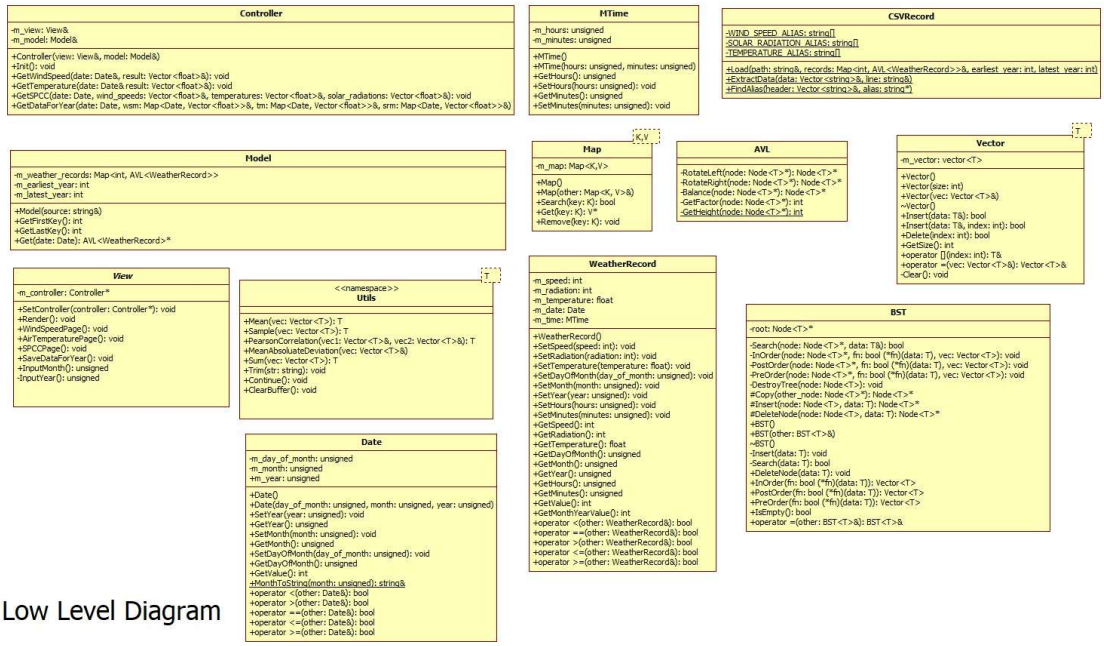


High Level Diagram

Version 1.0 - Added Vector, SDResult, View, Controller, Model, WeatherRecord, MTime, and Date

Association
Directed Association
Composition
Dependency

Below is a low-level diagram containing all the properties and methods of the classes.



Low Level Diagram

Data Dictionary

AVL Class

Name	Type	Access Modifier	Description	Rationale
RotateLeft(Node<T>* node)	Node<T>*	Private	Rotate a subtree to the left.	Rotates the subtree to the left if the tree is right-heavy.
RotateRight(Node<T>* node)	Node<T>*	Private	Rotate a subtree to the right.	Rotates the subtree to the right if the tree is left-heavy.
Balance(Node<T>* node)	Node<T>*	Private	Balance the AVL tree.	Ensures the AVL tree remains balanced after insertion or deletions. This guarantees logarithmic time complexity rather than linear time complexity.
GetFactor(Node<T>* node)	Int	Private	Get the balance factor of a node.	Get the balance factor of a node. The metric is used to determine whether rotations are needed to balance the tree.
GetHeight(Node<T>* node)	Int	Private	Get the height of a node.	Get the height of the node. It is used in conjunction with GetFactor method by subtracting the left and the right nodes.

BST Class

Name	Type	Access Modifier	Description	Rationale
BST()	Constructor (Default)	Public	Constructor to initialize an empty BST.	This constructor initializes the BST by setting nullptr to root.
BST(BST<T>& other)	Constructor (Copy)	Public	Copy constructor to create a new BST from another initialized BST.	Copies the data inside of another BST via deep copy.
~BST()	Destructor	Public	Destructor to clean up the BST.	Deletes all nodes that were initialized using new.
Search(Node<T>* node, T& data)	Bool	Private	Search for a node with the given data.	Searches a value by traversing left or right of the node. If the value does not match, it will traverse until it encounters a nullptr and returns false. Otherwise, it will return true.
InOrder(Node<T>* node, bool(*fn)(T& data), Vector<T>& vec)	Void	Private	Perform an in-order traversal of the BST.	Traverses the tree by visiting the left subtree, the current node, then the right subtree. This is used for sorting values.
PreOrder(Node<T>* node, bool(*fn)(T& data) , Vector<T>& vec)	Void	Private	Perform a pre-order traversal of the BST.	Traverses the tree from the, the current node, the left subtree, then the right subtree. This is used for copying the tree.
PostOrder(Node<T>* node, Bool(*fn)(T& data) , Vector<T>& vec	Void	Private	Perform a post-order traversal of the BST.	Traverses the tree from the left subtree, the right subtree, then the current node. This is used for deleting nodes.

DestroyTree(Node<T>* node)	Void	Private	Destroy the entire tree and free memory.	Destroys the entire tree by clearing all the memory.
Copy(Node<T>* other_node)	Node<T>*	Protected	Copy a node from another tree.	Copies the node from another tree. When an assignment or copy operator is used, this method allows the other node to be copied into the current tree.
DeleteNode(Node<T>* node, T& data)	Node<T>*	Protected	Delete a node that matches the value.	Deletes the node that matches the value by removing it from the BST.
Insert(T& data)	Void	Public	Insert a value into the BST.	Inserts a value into the BST. It provides external method to insert value without going through the process of Insert implementation.
Search(T& data)	Bool	Public	Search for a value in the BST.	Searches for a specific element by leveraging the BST to reduce search time.
DeleteNode(T& data)	Void	Public	Delete a node from the table.	Deletes a node by specifying the value to search for.
InOrder(bool (*fn)(T& data)	Vector<T>	Public	Perform an in-order traversal of the BST.	Retrieves the data in sorted order and call the function to each of the node found in the tree. The collated result are returned.
PostOrder(bool (*fn)(T& data)	Vector<T>	Public	Perform a post-order traversal of the BST.	Retrieves the data starting from the left subtree, the right subtree, and finally, the node. The

				collated result are returned.
PreOrder(bool (*fn)(T& data)	Vector<T>	Public	Perform a pre-order traversal of the BST.	Retrieves the data starting from the node, the left subtree, and then, the right subtree. The collated result are returned.
IsEmpty()	Bool	Public	Check whether the tree is empty.	Checks whether the root is empty which is only accessible internally.
m_root	Node<T>*	Private	The root node of the BST.	The root node of the BST is required to traverse through the node to find, delete, and insert value.

Controller Class

Name	Type	Access Modifier	Description	Rationale
Controller(View &view, Model &model)	Constructor (Parameterized)	Public	Constructor for the Controller Class.	The constructor initializes the Controller object with Model and View.
Init()	Void	Public	Initializes the Controller.	Serves as the controller's entry point by rendering the initial view.
GetWindSpeed(const Date& date, Vector<float>& result)	Void	Public	Gets the wind speed for a specific month and year.	Retrieves the wind speed from the Model for a specific month and year before returning the result back using a pass by reference.
GetTemperature(const Date& date, Vector<float>& result)	Void	Public	Gets the temperature for each month of a specific year.	Retrieves the temperature from the Model based on a specific year before returning the result back

				using a pass by reference.
GetSPCC(Date date, Vector<float>& wind_speeds, Vector<float>& temperatures, Vector<float>& solar_radiations)	Void	Public	Get the sample Pearson Correlation Coefficient from the loaded data.	Retrieves the sample Pearson Correlation Coefficient by populating the pass by reference vector.
GetDataForYear(Date date, Map<Date, Vector<float>>& wind_speed_map, Map<Date, Vector<float>>& temperature _map, Map<Date, Vector<float>>& solar_radiation _map)	Void	Public	Get the data for a specific year.	Retrieves the data for each month of a specific year and insert the data with Date as a key and Vector<float> as a value into a pass by reference map.
m_view	View&	Private	Handles rendering of the view.	Enables the controller to render pages.
m_model	Model&	Private	Manages the retrieval of data from the model.	Enables the controller to retrieve relevant weather data.

CSVRecord Class

Name	Type	Access Modifier	Description	Rationale
Load(string& path, Map<int, AVL<WeatherRecord>> records, int& earliest_year, int& latest_year)	Void	Public	Load weather records from a CSV file and populate the record map.	This method loads the record into a Map which is stored in Model.
ExtractData(Vector<string>& data, String& line)	Void	Private	Extract and parses each line into a vector string data.	Extracts data from a line and stores it inside a vector string for further processing.
FindAlias(Vector<string>& header, string* alias)	Int	Private	Finds the index of an alias in the headers vector.	Searches through the header for any of the alternative names.

WIND_SPEED_ALIAS[]	String	Private	Wind speed alias used in the CSV file.	A constant static property that stores different alias for wind speed. This property is only accessible to CSVRecord.
SOLAR_RADIATION_ALIAS[]	String	Private	Solar radiation alias used in the CSV file.	A constant static property that stores different alias for solar radiation. This property is only accessible to CSVRecord.
TEMPERATURE_ALIAS[]	String	Private	Air ambient temperature alias used in the CSV file	A constant static property that stores different alias for temperature. This property is only accessible to CSVRecord.

Date Class

Name	Type	Access Modifier	Description	Rationale
Date()	Constructor (Default)	Public	Default constructor for Date object.	The constructor initializes the Date object with default values.
Date(unsigned day_of_month, unsigned month, unsigned year)	Constructor (Parameterized)	Public	Parameterized constructor for Date object.	The constructor initializes the Date object with day, month, and year.
MonthToString(unsigned month)	Const string&	Public	Converts a month number to its string representation.	Allow the program to display the month name.
m_day_of_month	Unsigned	Private	The day of the month (1-31)	Represents the day of the month as a non-negative integer and applies modulo arithmetic to limit the range from 1 to 31.
m_month	Unsigned	Private	The month (1-12)	Represents the month as a non-negative

				integer and applies modulo arithmetic to limit the range from 1 to 12. Acts as a filter when processing weather record.
m_year	Unsigned	Private	The year.	Represents the year as a non-negative integer. Acts as a filter when processing weather record.

Model Class

Name	Type	Access Modifier	Description	Rationale
Model(const string& source)	Constructor (Parameterized)	Public	Default constructor for the Model object.	The constructor initializes the Model object and loads all the weather records from the specified source.
GetFirstKey()	Int	Public	Retrieve the first key in the model.	Get the first key. The method allows the controller method GetSPCC to decode the earliest year that was set.
GetLastKey()	Int	Public	Retrieve the last key in the model.	Get the last key. The method allows the controller method GetSPCC to decode the latest year that was set.
m_earliest_year	Int	Private	The earliest year in the weather records.	Allows the first key to be retrieved using GetFirstKey method.
m_latest_year	Int	Private	The latest year in the weather records.	Allows the last key to be retrieved using GetLastKey method.
m_weather_records	Map<int, AVL<WeatherRecord>>	Private	A map to hold weather records indexed by year and month.	Stores the weather record in an indexed map of AVL<WeatherRecord>

MTime Class

Name	Type	Access Modifier	Description	Rationale
MTime()	Constructor (Default)	Public	A default constructor for Time object.	The constructor initializes the Date object.
MTime(unsigned hours, unsigned minutes)	Constructor (Parameterized)	Public	Parameterized constructor for Time object.	Constructs Time object using the parameters, hours, and minutes.
m_hours	Unsigned	Private	Hours in a 24-hour format.	Completes the Time API and reserves for future use. Represents time in a 24-hours format. The modulo arithmetic is applied to ensure it stays between 0 and 23.
m_minutes	Unsigned	Private	Minutes ranging from 0 to 59	Completes the Time API and reserves for future use. Represents time as minutes. The modulo arithmetic is applied to ensure it stays between 0 and 59.

Utils Namespace

Name	Type	Access Modifier	Description	Rationale
Mean(Vector<T>& vec)	T	Public	Calculate the mean of the dataset.	Uses the data stored in the vector to process and return the average.
Sample(Vector<T>& vec)	T	Public	Calculate the sample standard deviation of a vector.	Uses the data stored in the vector to process and return the sample

				standard deviation.
PearsonCorrelation(Vector<T>& vec1, Vector<T>& vec2)	T	Public	Calculate the Sample Pearson Correlation Coefficient.	Uses vec1 and vec2 to calculate the SPCC and return the result.
MeanAbsoluteDeviation(Vector<T>& vec)	T	Public	Calculate the mean absolute deviation of a vector.	Uses the data stored in the vector to process and return the mean absolute deviation.
Sum(Vector<T>& vec)	T	Public	Calculate the sum of a vector.	Uses the data to calculate the sum and return the total of all values added.
Trim(string& str)	Void	Public	Trim leading and trailing whitespace from the string.	Trims the CSV header along with the data to prevent any comparison and conversion issue.
Continue()	Void	Public	Display a message to continue.	Informs the user that the page has finished displaying the result. This stops the page from displaying other pages immediately.
ClearBuffer()	Void	Public	Clear the input buffer.	Clears the input buffer such as newline after keying in values (e.g. double, float, string). If the user provides an invalid value, it will reset the error state.

Vector Class

Name	Type	Access Modifier	Description	Rationale
Vector()	Constructor (Default)	Public	Default constructor for	The default constructor object that

			the vector object.	allows variable to be declared without any parameter.
Vector(int size)	Constructor (Parameterized)	Public	Parametized constructor for the Vector object.	The constructor initializes Vector object and allocates the vector with the provided capacity.
Vector(const Vector<T>& vec)	Constructor (Copy Assignment)	Public	Copies from another Vector object.	The constructor copies from another vector object.
Insert(const T& data)	Bool	Public	Inserts data into the vector at the last index.	Inserts the data without providing the second parameter.
Insert(const T& data, int index)	Bool	Public	Inserts data into the vector at a specific index.	Allows for the insertion of items in different position.
Delete(int index)	Bool	Public	Deletes data from the vector at a specific index.	Allows for deletion of item.
GetSize()	Int	Public	Get the size of the vector.	Get the current size of the vector and determines if the vector contains any items.
operator [](int index)	Const T&	Public	Subscript operator to access elements in the vector as a read-only reference.	Enables the vector object to retrieve data using [i].
operator [](int index)	T&	Public	Subscript operator to access elements in the vector as a modifiable reference.	Enables the specific index to be modified using [i].
operator =(const Vector<T>& vec)	Vector<T>&	Public	Assignment operator to copy the contents of another vector.	Enables vector to assign data from another vector easily.
Clear()	Void	Public	Clear the vector by removing all elements.	Deletes the element inside the vector without redeclaring a

				vector to clear the data.
m_vector	T*	Private	Stores the vector data.	Stores the object inside the vector for further processing.

View Class

Name	Type	Access Modifier	Description	Rationale
SetController(const Controller* controller)	Void	Public	Sets the controller.	Assigns the controller to view without direct manipulation of the m_controller property.
Render(const string& page)	Void	Public	Renders the specific page.	Renders the available page in the if statement and display the list of available options.
WindSpeedPage()	Void	Private	Displays the average and sample standard deviation of the wind speed for a specific month and year.	Displays information about the wind speed after user enters the month and year.
AirTemperaturePage()	Void	Private	Displays the average and sample standard deviation of the ambient air temperature.	Displays information about the ambient air temperature for each month after user enters the year.
SPCCPage()	Void	Private	Display Sample Pearson Correlation Coefficient.	Displays information for a specific month based on the data collected.
SaveDataForYear()	Void	Private	Saves the average, mean absolute deviation, and sample standard deviation of the wind speed and ambient air temperature, the total solar radiation for each month into a file called “WindTempSolar.csv.”	Saves the data into a file called “WindTempSolar.csv”
InputMonth()	Unsigned	Private	Gets the input month from the user.	Validates the month input by checking that the value is between 1 and 12, rejecting alphabetic or out-of-range numbers, and preventing overflow.

				It informs the user on any invalid input.
InputYear()	Unsigned	Private	Gets the input year from the user.	Validates the year input by checking that the value is between 1 and 9999, rejecting alphabetic or out-of-range numbers, and preventing overflow. It informs the user on any invalid input.
m_controller	Controller*	Private	A pointer to the controller object.	Stores a reference to the controller object. It also allows view to fetch the required data via the controller.

WeatherRecord Class

Name	Type	Access Modifier	Description	Rationale
WeatherRecord()	Constructor (Default)	Public	Default constructor for WeatherRecord object.	The constructor initializes WeatherRecord object.
m_speed	Int	Private	Speed of the wind in m/s.	Represents speed as an integer. If speed is below 0, a sentinel value -9999 will replace the invalid value.
m_radiation	Int	Private	Solar radiation in W/m2	Represents solar radiation as an integer. The maximum recorded solar radiation is 1361 $W/m^{2,}$; therefore, value above 1500 W/m^2 is invalid. Similarly, if the values falls below 0, the sentinel value -9999 will replace the invalid value.
m_temperature	Float	Private	Temperature in degrees Celsius.	Represents air ambient temperature as an integer. The lowest temperature is

				around -89.2C, while the highest temperature is 56.7C. Valid values are capped between -100C to 100C. Any value outside the range is replaced by the sentinel value -9999.
m_date	Date	Private	Date of the weather record.	Holds the date object. The object is used to validate the specific date by checking the month and year.
m_time	MTime	Private	Time of the weather record.	Holds the time object. Reserved for future use.

Map Class

Name	Type	Access Modifier	Description	Rationale
Map()	Constructor (Default)	Public	Default constructor for Map object.	The constructor initializes Map object.
Map(const Map<K, V>& other)	Constructor (Copy)	Public	Copy constructor for Map object	Copies the data inside of another Map using std::map assignment operator.
Insert(K& key, V& value)	Void	Public	Insert a key-value pair into map.	Assigns the value with a key in the map object.
Search	Bool	Public	Search for a key in the map.	Finds the key that matches the provided value. The search will return either true if the key exist or false if no key is found.
Get	V*	Public	Retrieve the value associated with a key.	Return the value that is associated with the key. If no value is found,

				a nullptr is returned.
Remove	Void	Public	Remove a key-value pair from the map.	Removes the key-value pair that matches the selected key.

Rationale for Map<int, AVL<WeatherRecord>

The program uses Map<int, AVL<WeatherRecord>> to structure weather data by month and year, and then by timestamp within that month.

```
((GetYear() - 1970) * 372 + GetMonth() * 31)
```

The design of this formula ensures that varying month lengths are abstracted away by treating each month uniformly as 31 days, eliminating the need to calculate the actual number of days in each month. Another benefit is that it accommodates year before 1970, as the key can be negative.

The decision to use AVL lies in its ability to self-rebalance after insertion. This ensures that both insertion and lookup operations to remain efficient. Since it stores WeatherRecord objects, the overloaded comparison operator uses the following formula to impose the temporal ordering:

```
((GetMonthYearValue() + (GetDayOfMonth() - 1)) * 24 * 60) +
(GetHours() * 60 + GetMinutes())
```

Despite its efficiency, the chosen representation introduces certain drawbacks. By integrating the day of month into the timestamp, it becomes difficult to retrieve individual date component directly from the AVL tree. One possible solution is to introduce a secondary map nested within the outer map, which would increase overhead but reduce the number of data elements within each AVL tree.

```
Map<int, Map<int, AVL<WeatherRecord>>>
```

Another alternative is to invert the hierarchy, using the AVL tree for month-year lookup while the inner map manages day and time. This becomes a viable option but introduces an issue where the nodes becomes larger, leading to increased overhead during rebalancing, as each balancing operation involves copying or moving data structures.

Test Plans

The test plan contains Unit Test, Integration Test, Application Test, and edge cases. Unit Test and Integration Test can be found in test folder.

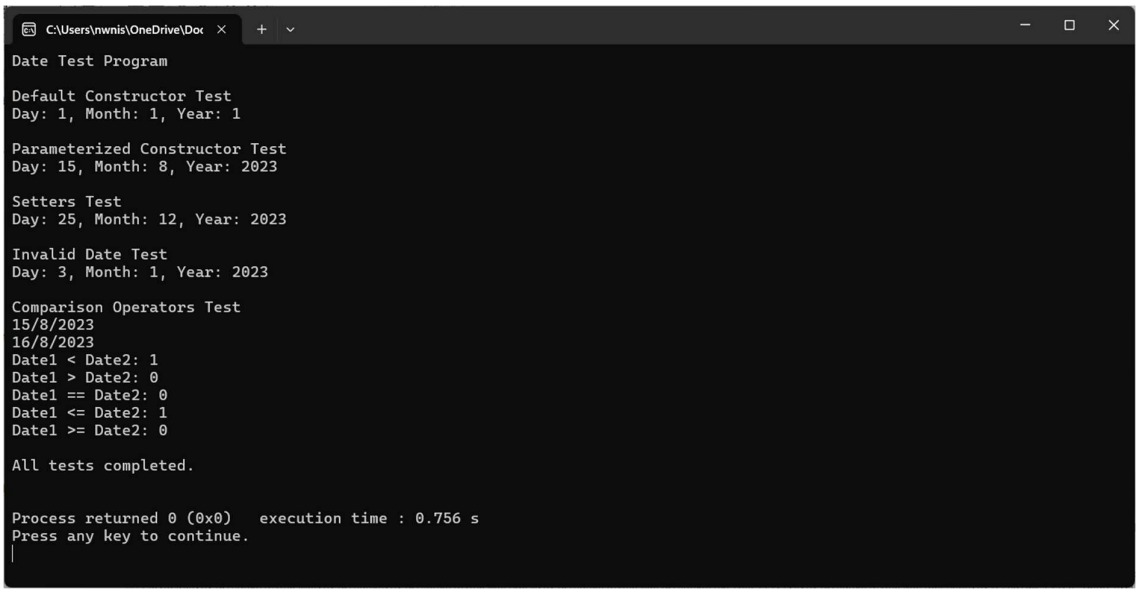
Date Class (Unit Test)

ID	Description	Input	Expected Output	Outcome
1	Create a date object without any parameter.	Date date; cout << "Day: " << date.GetDayOfMonth() << ", Month: " << date.GetMonth() << ", Year: " << date.GetYear() << endl;	Day: 1, Month: 1, Year: 1	Pass
2	Create a date with parameters.	Date date(15, 8, 2023); cout << "Day: " << date.GetDayOfMonth() << ", Month: " << date.GetMonth() << ", Year: " << date.GetYear() << endl;	Day: 15, Month: 8, Year: 2023	Pass

3	Set the date object with setter methods.	Date date;
		date.SetDayOfMonth(25);
		date.SetMonth(12);
		date.SetYear(2023);
		
		cout << "Day: " << date.GetDayOfMonth() << ", Month: " << date.GetMonth() << ", Year: " << date.GetYear() << endl;
			Day: 25, Month: 12, Year: 2023	Pass

4	Invalid date.	Date date(35, 13, 2023); cout << "Day: " << date.GetDayOfMonth() << ", Month: " << date.GetMonth() << ", Year: " << date.GetYear() << endl;	Day: 3, Month: 1, Year: 2023	Pass

5	Compares date with another date.	Date date1(15, 8, 2023);
		Date date2(16, 8, 2023);
		
		cout << date1.GetDayOfMonth() << '/' << date1.GetMonth() << '/' << date1.GetYear() << endl;
		cout << date2.GetDayOfMonth() << '/' << date2.GetMonth() << '/' << date2.GetYear() << endl;
		cout << "Date1 < Date2: " << (date1 < date2) << endl;
		cout << "Date1 > Date2: " << (date1 > date2) << endl;
		cout << "Date1 == Date2: " << (date1 == date2) << endl;
		cout << "Date1 <= Date2: " << (date1 <= date2) << endl;
		cout << "Date1 >= Date2: " << (date1 >= date2) << endl;
			15/8/2023
			16/8/2023
			Date1 < Date2: 1
			Date1 > Date2: 0
			Date1 == Date2: 0
			Date1 <= Date2: 1
			Date1 >= Date2: 0
				Pass

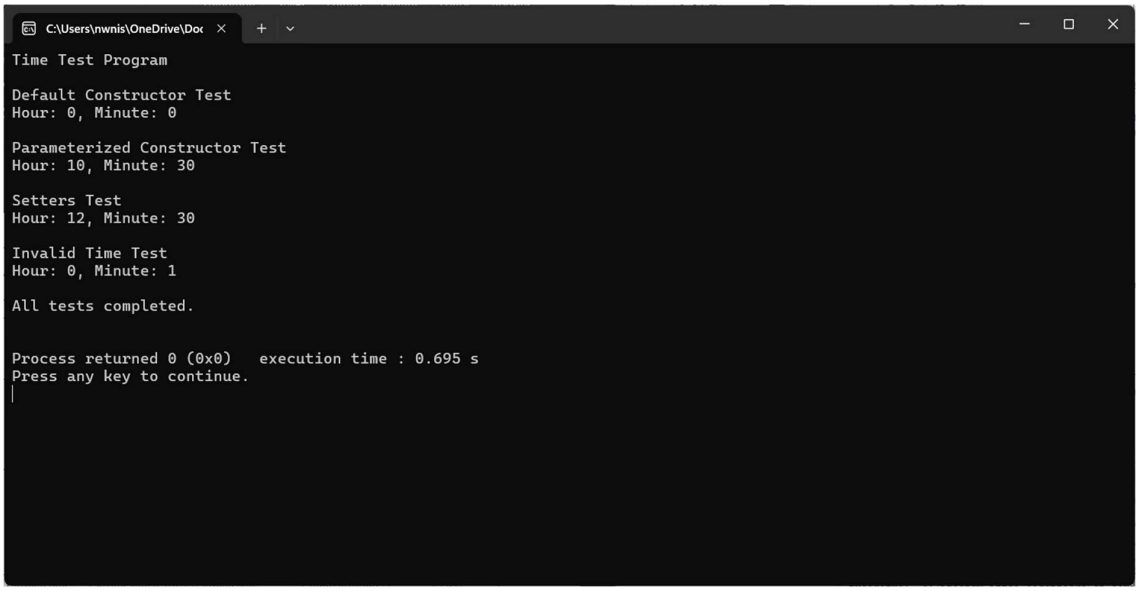


MTime Class (Unit Test)

ID	Description	Input	Expected Output	Outcome
1	Create a time object without any parameter.	MTime time; cout << "Hour: " << time.GetHours() << ", Minute: " << time.GetMinutes() << endl;	Hour: 0, Minute: 0	Pass
2	Create a time object with parameters.	MTime time(10, 30); cout << "Parameterized Constructor Test" << endl; cout << "Hour: " << time.GetHours() << ", Minute: " << time.GetMinutes() << endl;	Hour: 10, Minute: 30	Pass

3	Set the time object with setter methods.	MTime time;
		time.SetHours(12);
		time.SetMinutes(30);
		
		cout << "Hour: " << time.GetHours() << ", Minute: " << time.GetMinutes() << endl;
			Hour: 12, Minute: 30	Pass

4	Invalid time.	MTime time(24, 61); // Invalid time
		
		cout << "Hour: " << time.GetHours() << ", Minute: " << time.GetMinutes() << endl;
			Hour: 0, Minute: 1	Pass



Vector Class (Unit Test)

ID	Description	Input	Expected Output	Outcome
1	Create a vector object without any parameter.	Vector<int> vec; cout << "Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;	Size: 0, Capacity: 10	Pass
2	Create a vector object with an invalid size.	Vector<int> vec(-10); cout << "Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;	Size: 0, Capacity: 10	Pass

3	Create a vector object with a valid size.	Vector<int> vec(5);
		
		cout << "Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;
		
		for (int i = 0; i < 5; ++i) {
		  vec.Insert(i);
		}
		
		cout << "After inserting elements: Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;
			Size: 0, Capacity: 5
			After inserting elements: Size: 5, Capacity: 10
				Pass

4	Insert numbers into a vector object.	Vector<int> vec(2);
		
		vec.Insert(1);
		vec.Insert(2);
		vec.Insert(3);
		
		cout << "After inserting elements: Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;
			After inserting elements: Size: 3, Capacity: 4
			Element at index 0: 1
			Element at index 1: 2
			Element at index 2: 3
				Pass

		for (int i = 0; i < vec.GetSize(); ++i) {
		  cout << "Element at index " << i << ":
		" << vec[i] << endl;
		  }		

5	Insert numbers at specific indices.	Vector<int> vec(5);
		
		vec.Insert(10, 0);
		vec.Insert(20, 1);
		vec.Insert(30, 2);
		
		cout << "After inserting elements: Size:
		" << vec.GetSize() << ", Capacity: " <<
		vec.GetCapacity() << endl;
		
		for (int i = 0; i < vec.GetSize(); ++i) {
		  cout << "Element at index " << i << ":
		" << vec[i] << endl;
		}
			After inserting elements: Size: 3, Capacity: 5
			Element at index 0: 10
			Element at index 1: 20
			Element at index 2: 30
				Pass

6	Insert numbers at specific indices twice.	Vector<int> vec(5);
		
		vec.Insert(1, 0);
		vec.Insert(2, 1);
		vec.Insert(3, 2);
		vec.Insert(4, 3);
		vec.Insert(5, 4);
		vec.Insert(6, 2);
		
		for (int i = 0; i < vec.GetSize(); ++i) {
		  cout << "Element at index " << i << ":
		" << vec[i] << endl;
		}
			Element at index 0: 1
			Element at index 1: 2
			Element at index 2: 6
			Element at index 3: 3
			Element at index 4: 4
			Element at index 5: 5
				Pass

7	Insert numbers at specific indices with invalid index.	Vector<int> vec(5);
		
		if(!vec.Insert(10, -1)){
		  cout << "Failed to insert at index -1"
		<< endl;
		}
		
		if(!vec.Insert(20, 6)){
		  cout << "Failed to insert at index 6"
		<< endl;
		}
			Insert elements at specific indices with invalid index.
			Failed to insert at index -1.
			Failed to insert at index 6
				Pass

8	Delete number from the vector.	Vector<int> vec(5);
		
		vec.Insert(1);
		vec.Insert(2);
		vec.Insert(3);
		
		cout << "Before deletion: Size: " <<
		vec.GetSize() << ", Capacity: " <<
		vec.GetCapacity() << endl;
		for (int i = 0; i < vec.GetSize(); ++i) {
		  cout << "Element at index " << i << ":
		" << vec[i] << endl;
		}
		vec.Delete(1);
		
		  cout << "After deleting element at
		index 1: Size: " << vec.GetSize() << ",
			Before deletion: Size: 3, Capacity: 5
			Element at index 0: 1
			Element at index 1: 2
			Element at index 2: 3
			After deleting element at index 1: Size: 2, Capacity: 5
			Element at index 0: 1
			Element at index 1: 3
				Pass

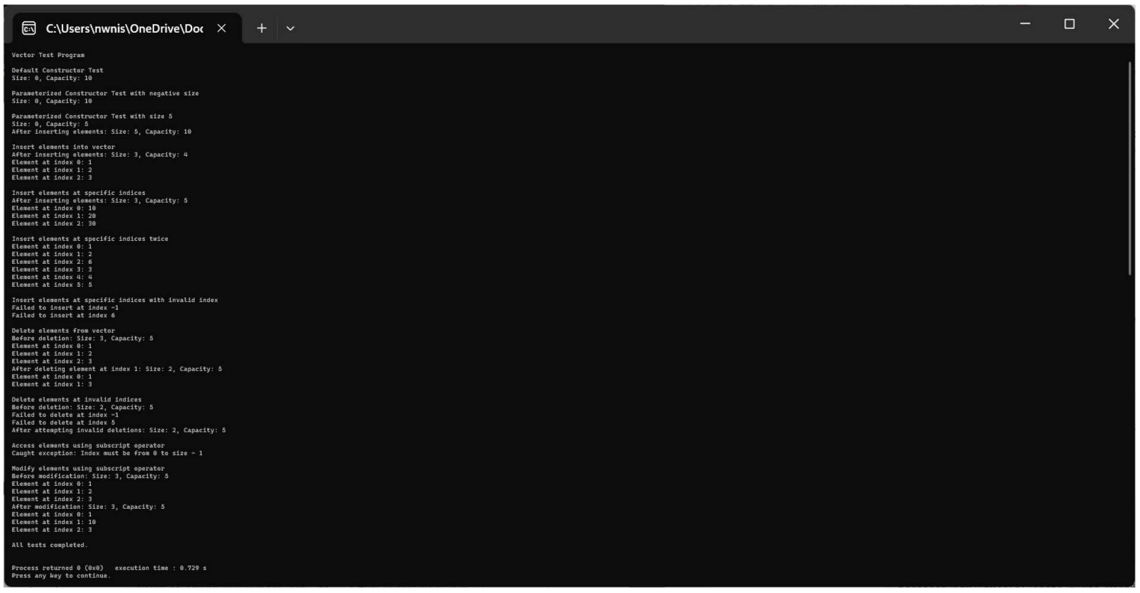
		Capacity: " << vec.GetCapacity() << endl;
		
		for (int i = 0; i < vec.GetSize(); ++i) {
		cout << "Element at index " << i << ": " << vec[i] << endl;
		}		

9	Delete number at invalid indices.	Vector<int> vec(5);
		
		vec.Insert(1);
		vec.Insert(2);
		
		cout << "Before deletion: Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;
		
		if (!vec.Delete(-1)) {
		cout << "Failed to delete at index -1" << endl;
		}
		
		if (!vec.Delete(5)) {
		cout << "Failed to delete at index 5" << endl;
		}
		
		cout << "After attempting invalid deletions: Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;
			Before deletion: Size: 2, Capacity: 5
			Failed to delete at index -1.
			Failed to delete at index 5.
			After attempting invalid deletions: Size: 2, Capacity: 5
				Pass

10	Access number using subscript operator.	Vector<int> vec(5);
		
		try{
		vec[3] = 10;
		}
		catch(std::out_of_range& e){
		cout << "Caught exception: " << e.what() << endl;
		}
			Caught exception: Index must be from 0 to size – 1	Pass

11	Modify elements using subscript operator.	Vector<int> vec(5);
		vec.Insert(1);
		vec.Insert(2);
		vec.Insert(3);
		
		cout << "Before modification: Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;
		for (int i = 0; i < vec.GetSize(); ++i) {
		cout << "Element at index " << i << ": " << vec[i] << endl;
		}
		
		try{
		vec[1] = 10;
		}
		catch(std::out_of_range& e){
		cout << "Caught exception: " << e.what() << endl;
		}
			Before modification:
			Size: 3, Capacity: 5
			Element at index 0: 1
			Element at index 1: 2
			Element at index 2: 3
			After modification:
			Size: 3, Capacity: 5
			Element at index 0: 1
			Element at index 1: 10
			Element at index 2: 3
				Pass

		cout << "After modification: Size: " << vec.GetSize() << ", Capacity: " << vec.GetCapacity() << endl;
		for (int i = 0; i < vec.GetSize(); ++i) {
		  cout << "Element at index " << i << ": " << vec[i] << endl;
		}		

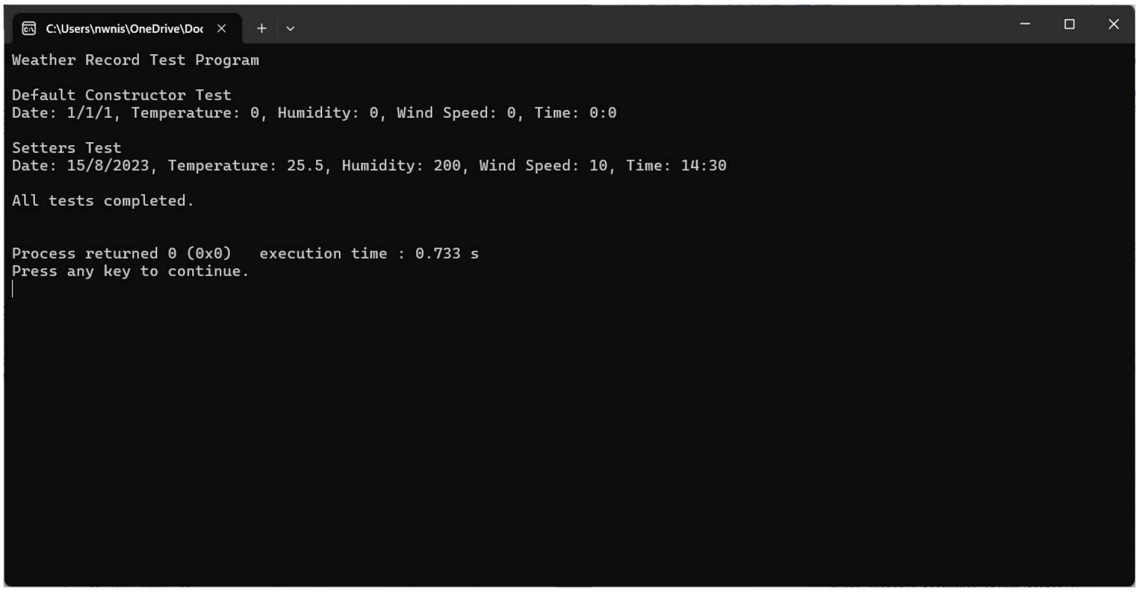


WeatherRecord Class (Integration Test)

ID	Description	Input	Expected Output	Outcome
1	Create a weather record object without any parameter.	WeatherRecord record; cout << "Date: " << record.GetDayOfMonth() << "/" << record.GetMonth() << "/" << record.GetYear() << ", Temperature: " << record.GetTemperature() << ", Humidity: " << record.GetRadiation() << ", Wind Speed: " << record.GetSpeed() << ", Time: " << record.GetHours() << ":" << record.GetMinutes() << endl;	Date: 1/1/1, Temperature: 0, Humidity: 0, Wind Speed: 0, Time: 0:0	Pass

2	Set the weather record object with setter methods.	WeatherRecord record;
		record.SetSpeed(10);
		record.SetRadiation(200);
		record.SetTemperature(25.5f);
		record.SetDayOfMonth(15);
		record.SetMonth(8);
		record.SetYear(2023);
		record.SetHours(14);
		record.SetMinutes(30);
			Date: 15/8/2023, Temperature: 25.5, Humidity: 200, Wind Speed: 10, Time: 14:30	Pass

		cout << "Date: " << record.GetDayOfMonth() << "/" << record.GetMonth() << "/" << record.GetYear() << ", Temperature: " << record.GetTemperature() << ", Humidity: " << record.GetRadiation() << ", Wind Speed: " << record.GetSpeed() << ", Time: " << record.GetHours() << ":" << record.GetMinutes() << endl;		



Map Test (Unit Test)

ID	Description	Input	Expected Output	Outcome

1	Insert number into a map.	Map<int, string> map;
		
		map.Insert(1, “one”);
		map.Insert(2, “two”);
		map.Insert(3, “three”);
		
		cout << "Map 1: " << *map.Get(1) << endl;
		cout << "Map 2: " << *map.Get(2) << endl;
			Map 1: One
			Map 2: Two
			Map 3: Three
				Pass

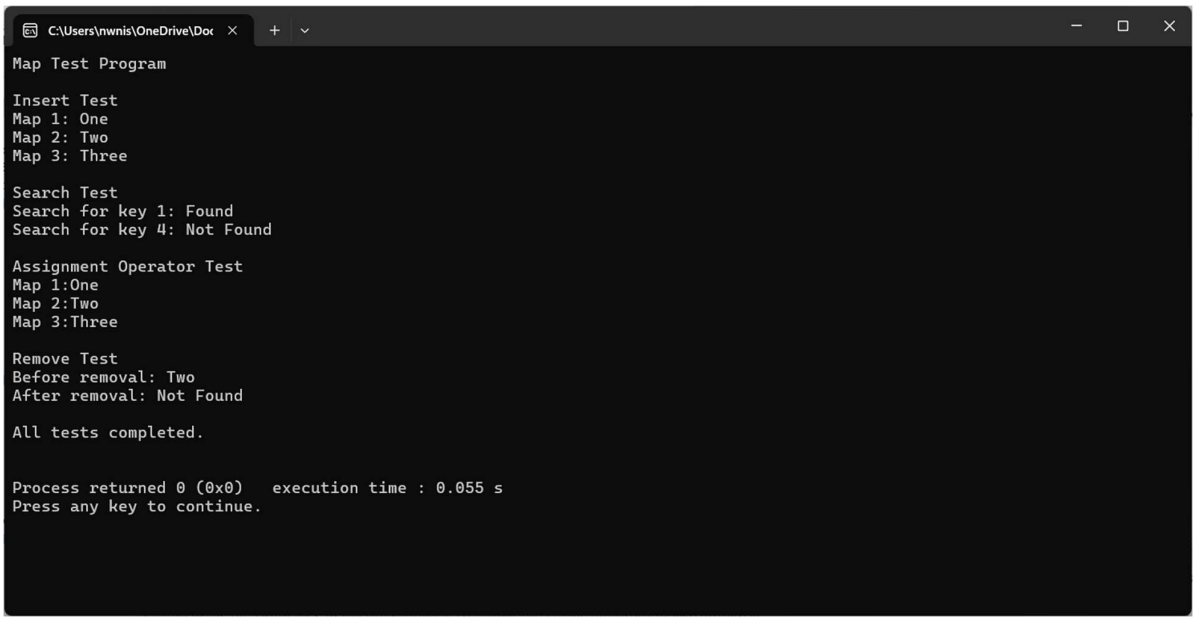
		cout << "Map 3: " << *map.Get(3) << endl;		

2	Search for an existing and non-existing number.	Map<int, string> map;
		
		map.Insert(1, "One");
		map.Insert(2, "Two");
		map.Insert(3, "Three");
		
		cout << "Search for key 1: " << (map.Search(1) ? "Found" : "Not Found") << endl;
		cout << "Search for key 4: " << (map.Search(4) ? "Found" : "Not Found") << endl;
			Search for key 1: Found
			Search for key 4: Not Found
				Pass

3	Assign a map to another map.	Map<int, string> map;
		Map<int, string> map_1;
		
		map.Insert(1, "One");
		map.Insert(2, "Two");
		map.Insert(3, "Three");
		
		map_1 = map;
		
		cout << "Map 1:" << map_1.Get(1) << endl;
		cout << "Map 2:" << map_1.Get(2) << endl;
		cout << "Map 3:" << map_1.Get(3) << endl;
			Map 1:One
			Map 2:Two
			Map 3:Three
				Pass

4	Remove number from a map.	Map<int, string> map;
		
		map.Insert(1, "One");
		map.Insert(2, "Two");
		map.Insert(3, "Three");
			Before removal: Two
			After removal: Not Found
				Passed

		cout << "Remove Test" << endl;
		cout << "Before removal: " << *map.Get(2) << endl;
		
		map.Remove(2);
		
		cout << "After removal: " << (map.Get(2) ? *map.Get(2) : "Not Found") << endl;		



BST Test (Unit Test)

ID	Description	Input	Expected Output	Outcome

1	Insert value into BST	BST<int> bst;
		
		bst.Insert(5);
		bst.Insert(3);
		bst.Insert(7);
		
		cout << "Is number 5 in BST? " << (bst.Search(5) ? "Yes" : "No") << endl;
		cout << "Is number 3 in BST? " << (bst.Search(3) ? "Yes" : "No") << endl;
		cout << "Is number 7 in BST? " << (bst.Search(7) ? "Yes" : "No") << endl;
		cout << "Is number 10 in BST? " << (bst.Search(10) ? "Yes" : "No") << endl;
			Is number 5 in BST? Yes
			Is number 3 in BST? Yes
			Is number 7 in BST? Yes
			Is number 10 in BST? No
				Pass

2	PreOrder traversal from BST.	BST<int> bst;
		
		bst.Insert(34);
		bst.Insert(23);
		bst.Insert(45);
		bst.Insert(12);
		bst.Insert(67);
		
		bst.PreOrder([](const int& data){
		  cout << data << " ";
		  return true;
		});
		
		cout << endl;
			34 23 12 45 67	Pass

3	InOrder traversal from BST.	BST<int> bst;
		
		bst.Insert(34);
		bst.Insert(23);
		bst.Insert(45);
		bst.Insert(12);
		bst.Insert(67);
		
		  bst.InOrder([](const int& data){
		  cout << data << " ";
		  return true;
		});
		
		cout << endl;
			12 23 34 45 67	Pass

4	PostOrder traversal from BST.	BST<int> bst;
		
		bst.Insert(34);
		bst.Insert(23);
		bst.Insert(45);
		bst.Insert(12);
		bst.Insert(67);
		
		bst.PostOrder([](const int& data){
		  cout << data << " ";
		  return true;
		});
		
		cout << endl;
			12 23 67 45 34	Pass

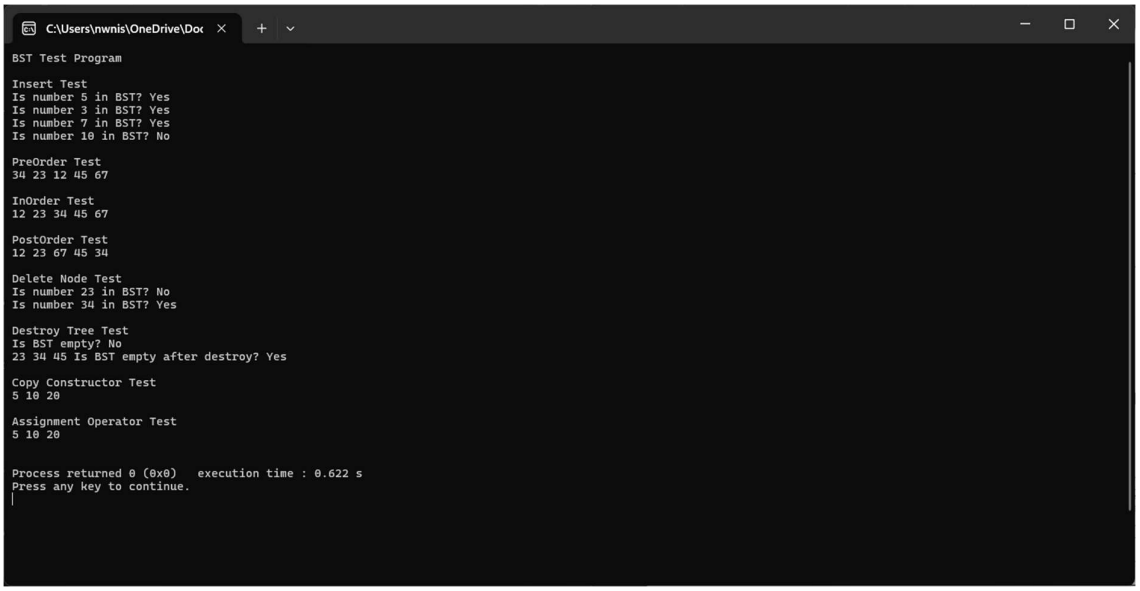
5	Delete node from the BST.	BST<int> bst;
		
		bst.Insert(34);
		bst.Insert(23);
		
		bst.DeleteNode(23);
		cout << "Is number 23 in BST? " <<
		(bst.Search(23) ? "Yes" : "No") << endl;
		cout << "Is number 34 in BST? " <<
		(bst.Search(34) ? "Yes" : "No") << endl;
			Is number 23 in BST? No
			Is number 34 in BST? Yes
				Pass

6	Destroy BST tree.	BST<int> bst; bst.Insert(34);	Destroy Tree Test Is BST empty? No	Pass

		bst.Insert(23);
		bst.Insert(45);
		
		cout << "Is BST empty? " << (bst.IsEmpty() ? "Yes" : "No") << endl;
		bst.InOrder([](const int& data){
		  cout << data << " ";
		  return true;
		});
		
		bst.DestroyTree();
		cout << "Is BST empty after destroy? " << (bst.IsEmpty() ? "Yes" : "No") << endl;
			23 34 45 Is BST empty after destroy? Yes	

7	Copy from another BST.	BST<int> bst;
		
		bst.Insert(10);
		bst.Insert(20);
		bst.Insert(5);
		
		BST<int> bst_copy(bst);
		
		bst_copy.InOrder([](const int& data){
		  cout << data << " ";
		  return true;
		});
		cout << endl;
			5 10 20	Pass

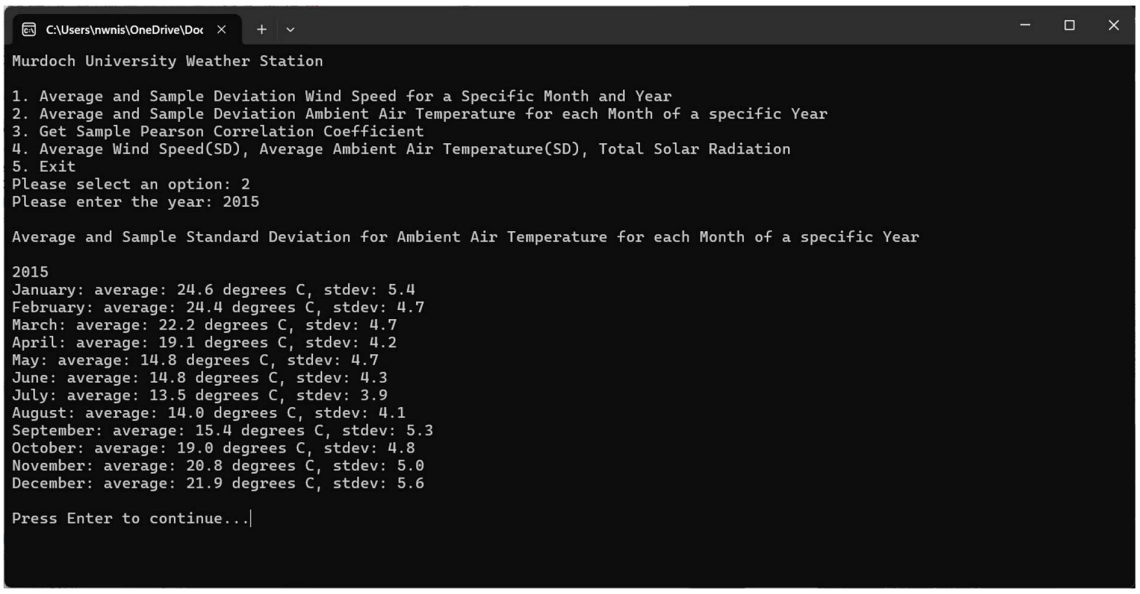
8	Assign BST with another BST.	BST<int> bst;
		
		bst.Insert(10);
		bst.Insert(20);
		bst.Insert(5);
		
		BST<int> bst_assignment_op;
		bst_assignment_op = bst;
		
		cout << "Assignment Operator Test" << endl;
		bst_assignment_op.InOrder([](const int& data){
		  cout << data << " ";
		  return true;
		});
		cout << endl;
			50 10 20	Pass



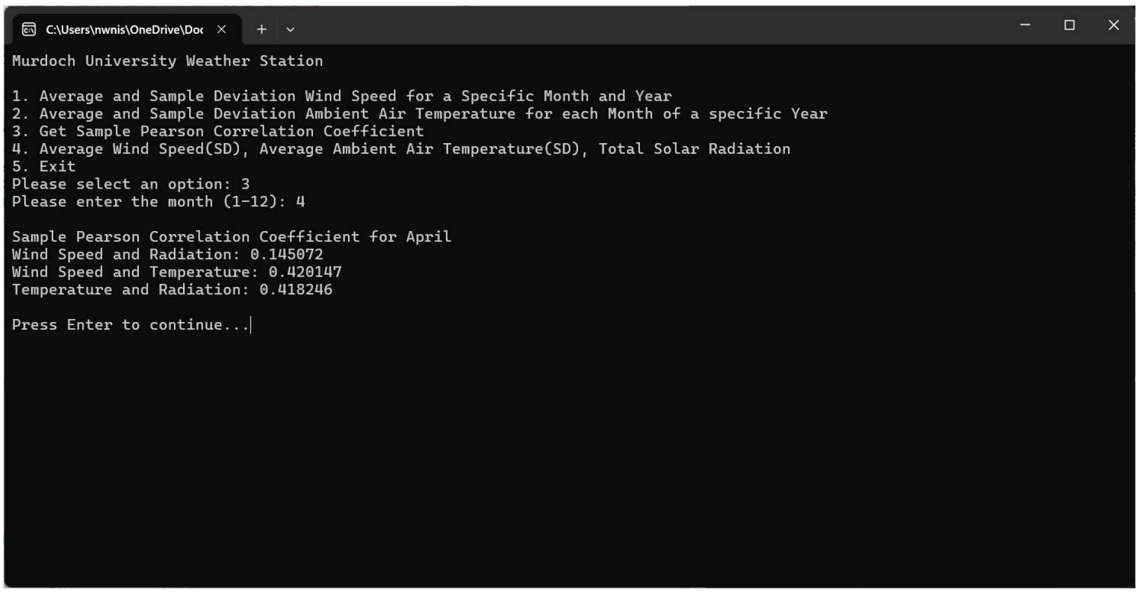
Application Test

Test ID	APP_PAGE_1
Description	Verify that the system calculates the average and standard deviation of wind speed based on user's input.
Precondition	CSV is loaded from data_source.txt.
Input	Please enter the month (1-12): 3 Please enter the year: 2015
Expected Result	The correct average and standard deviation of the wind speed in km/h are displayed.

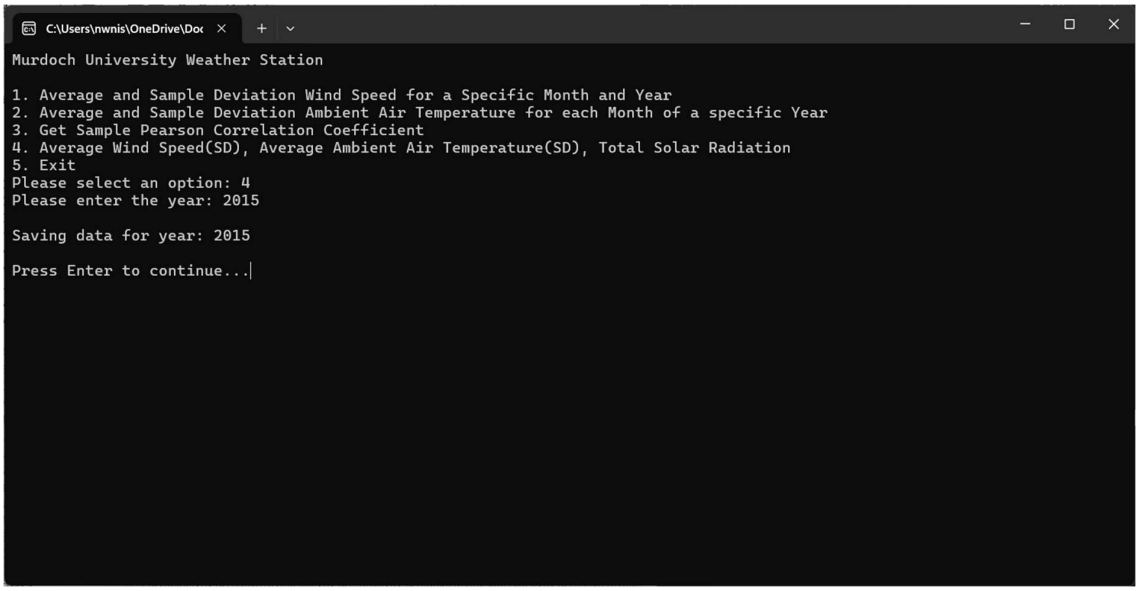
Test ID	APP_PAGE_2
Description	Verify that the system calculates the average and standard deviation of the temperature for each month of a specific year.
Precondition	CSV is loaded from data_source.txt.
Input	Please enter the year: 2015
Expected Result	The correct average and standard deviation of the temperature for all the months are displayed.

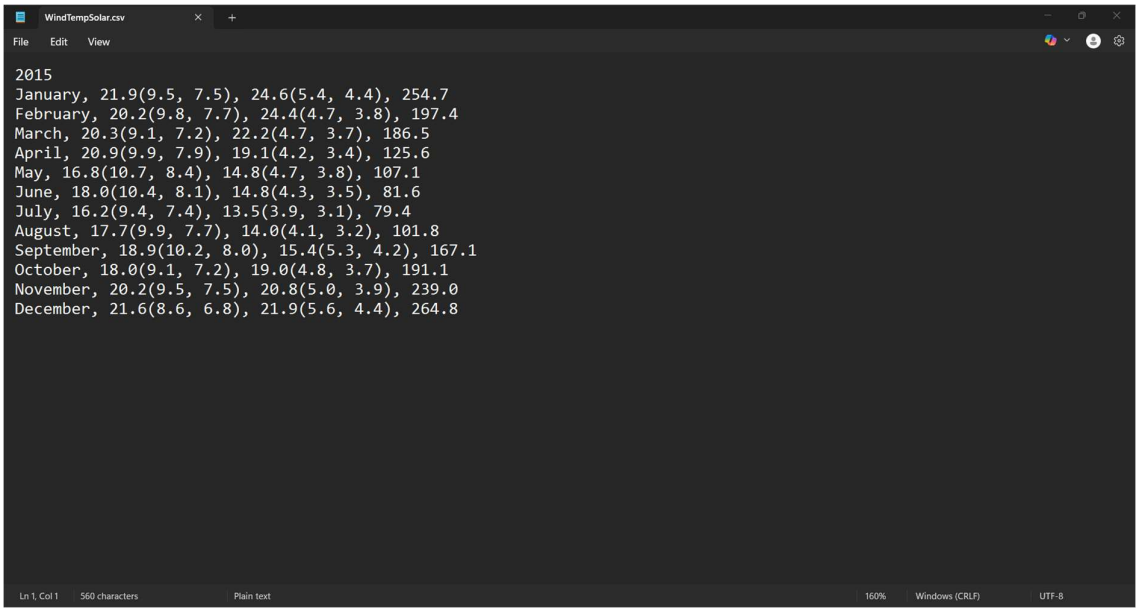


Test ID	APP_PAGE_3
Description	Verify that the system calculates sample Pearson correlation coefficient.
Precondition	CSV is loaded from data_source.txt.
Input	Please enter the month (1-12): 4
Expected Result	The correct Sample Pearson Correlation Coefficient for April.

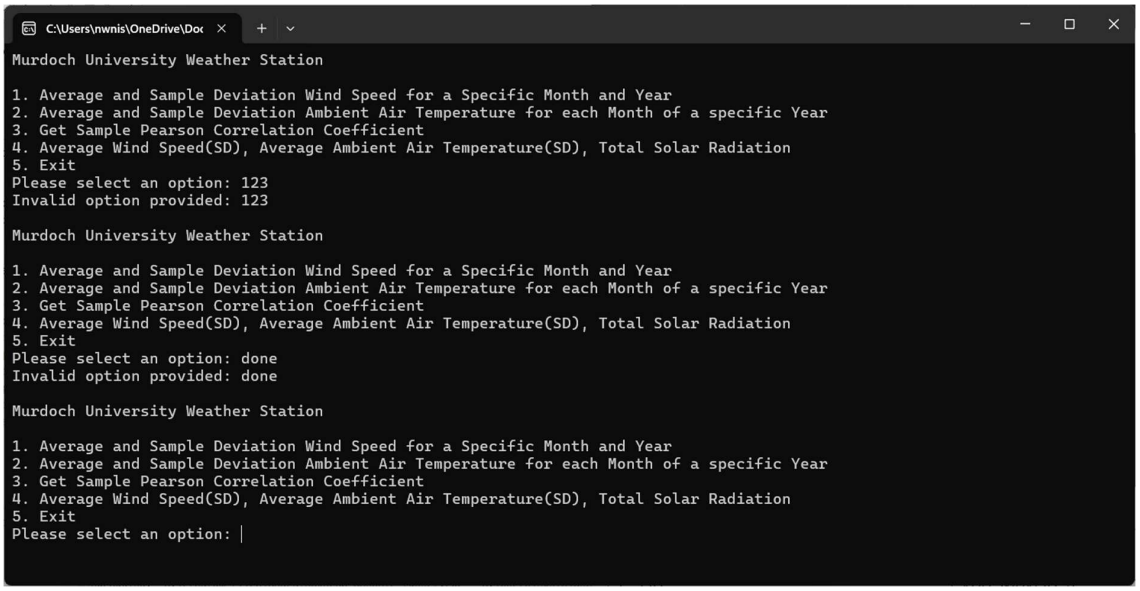


Test ID	APP_PAGE_4
Description	Verify that the system generates a file called WindTempSolar.csv. The file contains the average and sample standard deviation of the temperature, wind speed, and the total solar radiation for each month based on a specific year.
Precondition	CSV is loaded from data_source.txt.
Input	Please enter the year: 2015
Expected Result	A generated file containing the data for a specific year.

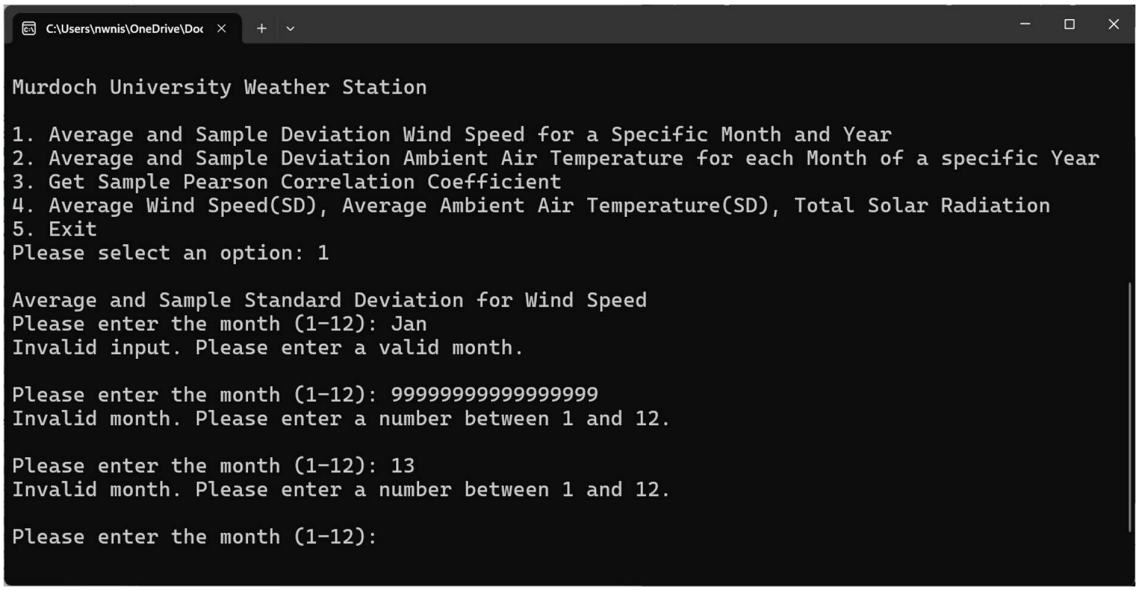




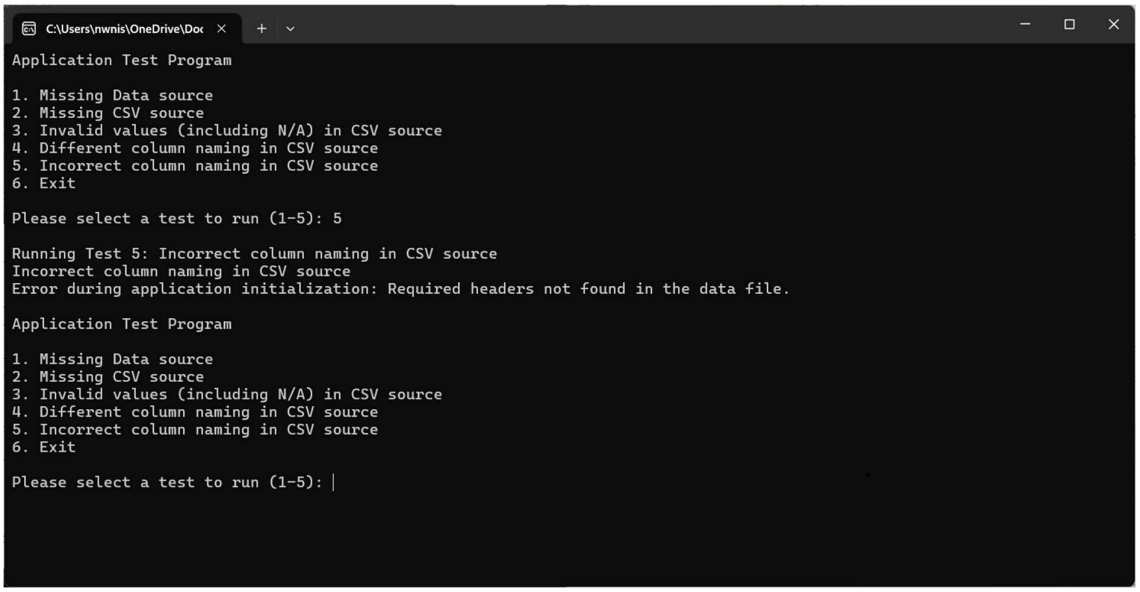
Test ID	APP_INVALID_1
Description	Verify that the menu ignores invalid input that are either out of range or non-numeric character.
Precondition	CSV is loaded from data_source.txt.
Input	Please select an option: 123 Please select an option done
Expected Result	Prints the menu if it encounters the error.



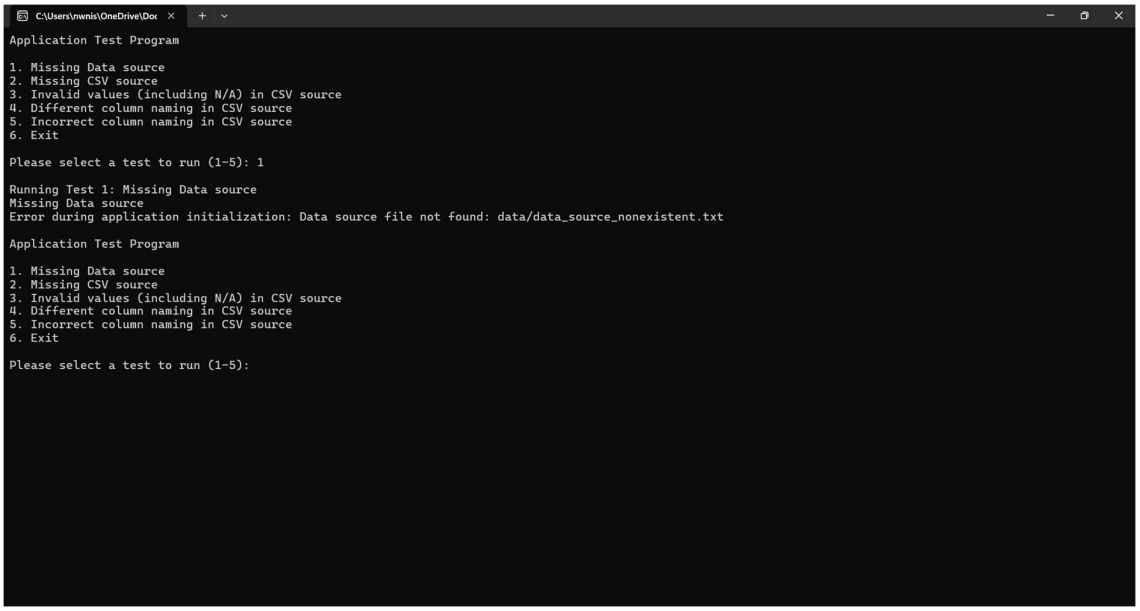
Test ID	APP_INVALID_2
Description	Verify that the month ignores invalid input that are: 1. Non-numeric character. 2. Out of the unsigned data type range. 3. Less than 1 or more than 12.
Precondition	CSV is loaded from data_source.txt.
Input	Please enter the month (1-12): Jan Please enter the month (1-12): 99999999999999999 Please enter the month (1-12): 13
Expected Result	Display the error message and ask the user to enter the valid month.



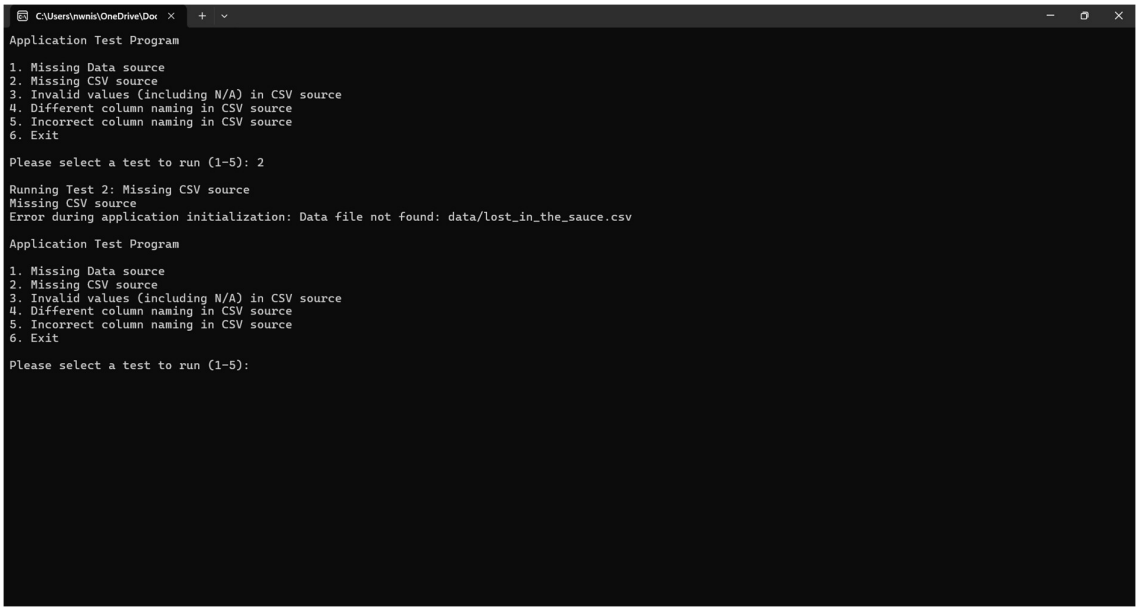
Test ID	APP_INVALID_3
Description	Verify that the system stops processing after the required is not found.
Precondition	CSV is loaded from data_source.txt.
Input	
Expected Result	Display the error message “Required headers not found.”



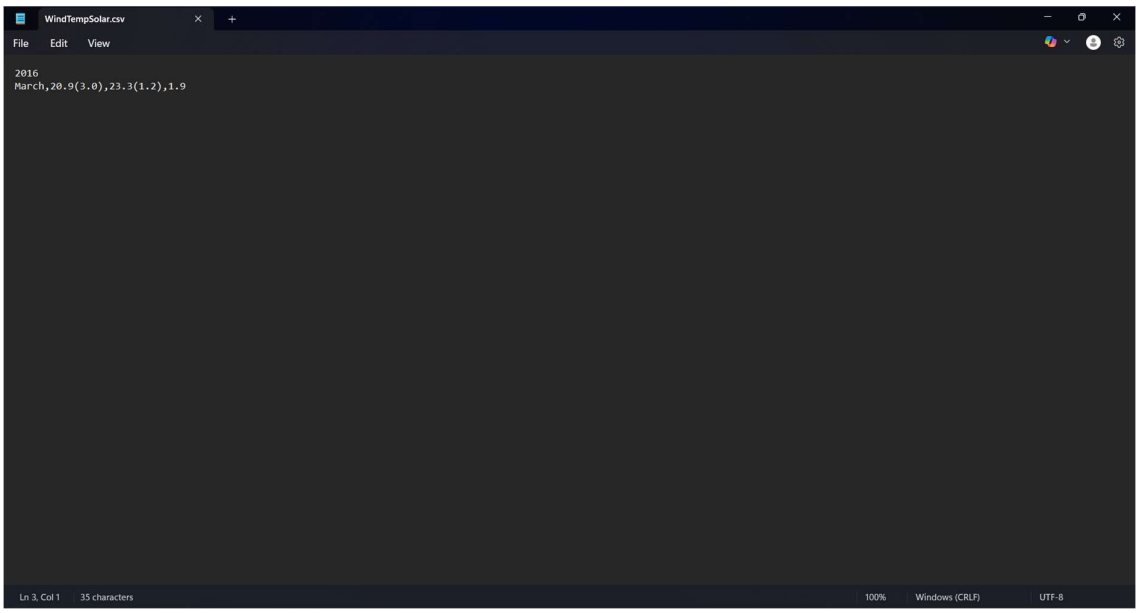
Test ID	APP_MISSING_SOURCE_1
Description	Verify that the system stops processing if the data_source.txt is not found.
Precondition	
Input	
Expected Result	Display the error message “Data source file not found.”



Test ID	APP_MISSING_SOURCE_2
Description	Verify that the system stops processing if the link inside data_source.txt is invalid.
Precondition	data_source.txt is loaded.
Input	
Expected Result	Display the error message “Data file not found.”



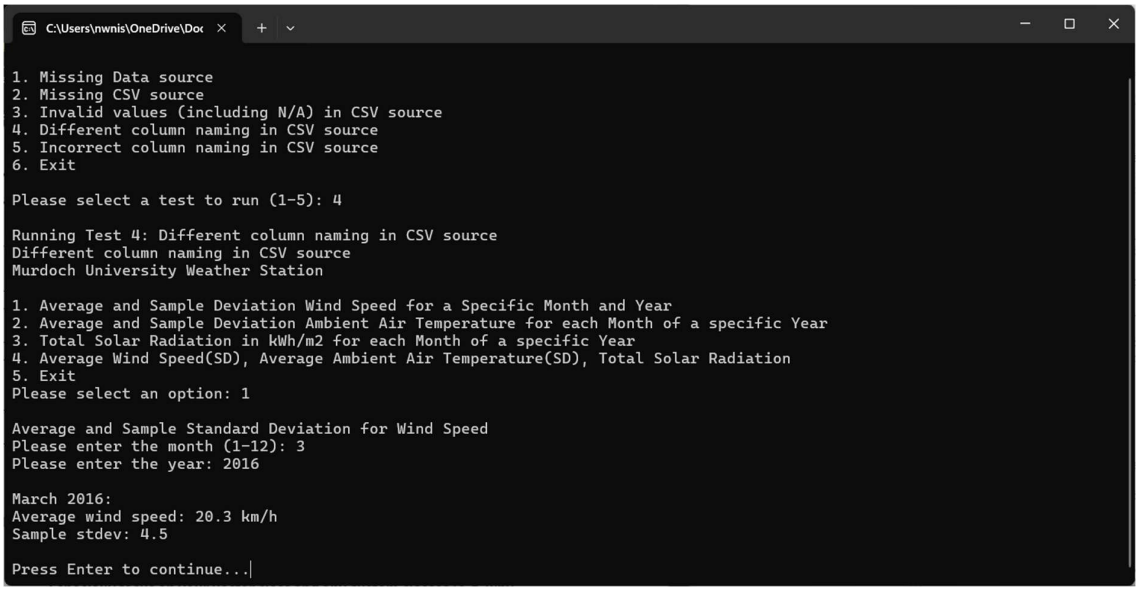
Test ID	APP_DATA_1
Description	Verify that the system can process rows containing partial or fully invalid data.
Precondition	CSV is loaded from data_source.txt.
Input	Please select an option: 4 Please enter the year: 2016
Expected Result	Calculates the data from columns while ignoring invalid columns.

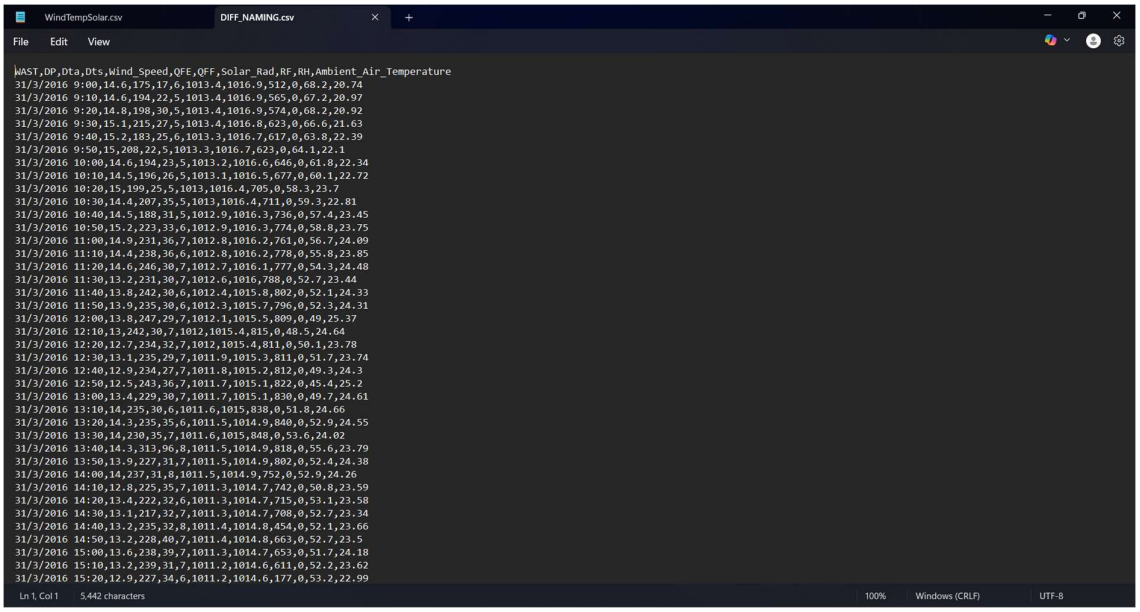


Note: The data comes from TEST_INVALID_VALUES.csv. The first row contains out

of range values, the second row contains invalid data (N/A, offline, NaN), and the third row contains an empty value along with two valid values.

Test ID	APP_DATA_2
Description	Verify that the system can process the CSV with different headers.
Precondition	CSV is loaded from data_source.txt.
Input	Please select an option: 1 Please enter the month (1-12): 3 Please enter the year: 2016
Expected Result	Calculates the data from columns while ignoring invalid columns.




WindTempSolar.csv
DIFF_NAMING.csv
File
Edit
View
WAST,DP,Dta,Dts,Wind_Speed,QFE,QFF,Solar_Rad,RF,RH,Ambient_Air_Temperature
31/3/2016 9:00,14.6,175,17,6,1013.4,1016.9,512,0,68.2,20.74
31/3/2016 9:10,14.6,194,22,5,1013.4,1016.9,565,0,67.2,20.97
31/3/2016 9:20,14.8,198,30,5,1013.4,1016.9,574,0,68.2,20.92
31/3/2016 9:30,15.1,215,27,5,1013.4,1016.8,623,0,66.6,21.63
31/3/2016 9:40,15.2,183,25,6,1013.3,1016.7,617,0,63.8,22.39
31/3/2016 9:50,15,208,22,5,1013.3,1016.7,623,0,64.1,22.1
31/3/2016 10:00,14.6,194,23,5,1013.2,1016.6,646,0,61.8,22.34
31/3/2016 10:10,14.5,196,26,5,1013.1,1016.5,677,0,60.1,22.72
31/3/2016 10:20,15,199,25,5,1013,1016.4,705,0,58.3,23.7
31/3/2016 10:30,14.4,207,35,5,1013,1016.4,711,0,59.3,22.81
31/3/2016 10:40,14.5,188,31,5,1012.9,1016.3,736,0,57.4,23.45
31/3/2016 10:50,15.2,223,33,6,1012.9,1016.3,774,0,58.8,23.75
31/3/2016 11:00,14.9,231,36,7,1012.8,1016.2,761,0,56.7,24.09
31/3/2016 11:10,14.4,238,36,6,1012.8,1016.2,778,0,55.8,23.85
31/3/2016 11:20,14.6,246,30,7,1012.7,1016.1,777,0,54.3,24.48
31/3/2016 11:30,13.2,231,30,7,1012.6,1016,788,0,52.7,23.44
31/3/2016 11:40,13.8,242,30,6,1012.4,1015.8,802,0,52.1,24.33
31/3/2016 11:50,13.9,235,30,6,1012.3,1015.7,796,0,52.3,24.31
31/3/2016 12:00,13.8,247,29,7,1012.1,1015.5,809,0,49,25.37
31/3/2016 12:10,13,242,30,7,1012,1015.4,815,0,48.5,24.64
31/3/2016 12:20,12.7,234,32,7,1012,1015.4,811,0,50.1,23.78
31/3/2016 12:30,13.1,235,29,7,1011.9,1015.3,811,0,51.7,23.74
31/3/2016 12:40,12.9,234,27,7,1011.8,1015.2,812,0,49.3,24.3
31/3/2016 12:50,12.5,243,36,7,1011.7,1015.1,822,0,45.4,25.2
31/3/2016 13:00,13.4,229,30,7,1011.7,1015.1,830,0,49.7,24.61
31/3/2016 13:10,14,235,30,6,1011.6,1015,838,0,51.8,24.66
31/3/2016 13:20,14.3,235,35,6,1011.5,1014.9,840,0,52.9,24.55
31/3/2016 13:30,14,230,35,7,1011.6,1015,848,0,53.6,24.02
31/3/2016 13:40,14.3,313,96,8,1011.5,1014.9,818,0,55.6,23.79
31/3/2016 13:50,13.9,227,31,7,1011.5,1014.9,802,0,52.4,24.38
31/3/2016 14:00,14,237,31,8,1011.5,1014.9,752,0,52.9,24.26
31/3/2016 14:10,12.8,225,35,7,1011.3,1014.7,742,0,50.8,23.59
31/3/2016 14:20,13.4,222,32,6,1011.3,1014.7,715,0,53.1,23.58
31/3/2016 14:30,13.1,217,32,7,1011.3,1014.7,708,0,52.7,23.34
31/3/2016 14:40,13.2,235,32,8,1011.4,1014.8,454,0,52.1,23.66
31/3/2016 14:50,13.2,228,40,7,1011.4,1014.8,663,0,52.7,23.5
31/3/2016 15:00,13.6,238,39,7,1011.3,1014.7,653,0,51.7,24.18
31/3/2016 15:10,13.2,239,31,7,1011.2,1014.6,611,0,52.2,23.62
31/3/2016 15:20,12.9,227,34,6,1011.2,1014.6,177,0,53.2,22.99
Ln 1, Col 1
5,442 characters
100%
Windows (CRLF)
UTF-8